Encrypted DNS Resolver

Project Stage II: Final Report

Dhruv Divyanshu Gupta Arnav Bhate

29 October 2026

Contents

1 Abstract **2**

2 Problem definition and requirements **3**
2.1 Scope and assumptions . . . 3
2.2 Functional requirements . . . 3
2.3 Security requirements . . . 4
2.4 Assets, trust boundaries and adversaries . . . 4
2.5 Out of scope: what the prototype does not protect . . . 5

3 Architecture and design **5**
3.1 Components and interfaces . . . 5
3.2 Life of one query (end-to-end data flow) . . . 6
3.3 Where security checks occur . . . 7
3.4 Error handling . . . 7
3.5 Key design choices and reasoning . . . 8

4 Implementation **8**
4.1 Language, libraries and environment . . . 8
4.2 Modules and reuse . . . 9
4.3 Important algorithms . . . 9
4.4 Configuration and integration . . . 10
4.5 Implementation challenges . . . 10

5 Testing, results and security evaluation **10**
5.1 Test environment . . . 10
5.2 Automated test suite . . . 11
5.3 Test cases: expected versus actual . . . 11
5.4 Evidence: a DNS query completing over DNS-over-TLS . . . 13
5.5 Evidence: upstream certificate validation . . . 15
5.6 Evidence: invalid responses, upstream failure, timeouts and recovery . . . 16
5.7 Evidence: availability under load and abuse . . . 18
5.8 Security evaluation . . . 20
5.9 Defects discovered . . . 21

6 Limitations and lessons learned **21**
6.1 Achieved requirements . . . 21
6.2 Known weaknesses . . . 21
6.3 Lessons learned . . . 22

6.4 Five things to remember about this project 22

7 Deployment and future work **23**
7.1 Dependencies and build 23
7.2 Run 23
7.3 Tests 23
7.4 Repository structure 23
7.5 Most important next improvements 24

8 Appendix: minimum technical evidence index **24**

1 Abstract

Ordinary DNS travels in plain text over UDP port 53. Anyone on the path (a shared Wi-Fi network, an ISP, a campus firewall) can read every name a device looks up, and can forge answers. We built a DNS resolver that clients reach over **DNS-over-TLS** (DoT, RFC 7858). Since Stage I it can also talk **DoT to its upstream resolvers**, checking each upstream's certificate, so a query is encrypted from the client to the resolver that answers it.

The prototype is about 9,000 lines of C++23 on Linux, using OpenSSL 3 and nothing else at run time. It has three layers:

- **Front end.** A single-threaded epoll DoT server. It does the TLS 1.3 handshake, framing, bounds-checked parsing, policy checks and padding to 468-byte blocks.
- **Middleware.** Rate limiting, abuse blocking, request coalescing, deadlines, bounded queues, load balancing and per-worker circuit breakers. Every query is answered exactly once.
- **Backend.** A resolver chain: local records, a shared TTL cache, forwarding over UDP/TCP or DoT with certificate validation, and a full iterative recursive resolver.

A loopback-only web dashboard shows live metrics and runs the Lookup, Compare, Traffic, Load, Abuse, Failures and Certificates tests against the real server.

What works in the final prototype.

- `dig +tls` gets correct answers (NOERROR, NXDOMAIN, local records, REFUSED for blocked domains). A packet capture of both hops shows only TLS 1.3 records: no packet decodes as DNS, and no queried name appears anywhere in the capture. Messages are padded on both hops, so names of different lengths give records of identical size.
- Upstream certificates are validated on every new connection: chain to a trusted CA, name or IP, and validity dates. Wrong-name, wrong-IP, expired, self-signed and untrusted certificates are all rejected. The question is never sent to a rejected server, and the resolver never falls back to plain DNS.
- Invalid upstream responses (garbage, wrong ID, truncated, cut off mid-frame), upstream failure (closed port, server stopped, plain server where TLS was expected) and timeouts all become a bounded SERVFAIL, and the service recovers as soon as the upstream returns.
- 534 automated tests pass in release, AddressSanitizer+UBSan and ThreadSanitizer builds. 54 of them cover the upstream DoT client and certificate validation; others cover invalid responses and timeouts.
- Under load the middleware delivers exactly one reply per query (100,000-query flood, also with 20% injected failures). A paced 720 q/s run gives p99 10 ms with zero errors. An abusive client is blocked within 0.09 s while a polite one is never refused.

2 Problem definition and requirements

2.1 Scope and assumptions

The prototype is a *stub-facing recursive/forwarding resolver*: it sits between end devices and the wider DNS. In scope:

- the client hop (device to our resolver) over DoT;
- the upstream hop (our resolver to a public resolver) over DoT;
- resolver-side availability and input safety;
- local policy (blocked domains, local names);
- an operator dashboard.

Assumptions:

A1. The resolver host and its operator are trusted. The private key is only readable by the server process (mode 600, git-ignored).

B2. Clients are configured to use our resolver. For the prototype they accept our self-signed certificate, as `dig +tls` does by default. A deployment would use a CA-issued certificate.

C3. Upstream resolvers that pass certificate validation answer honestly (no DNSSEC validation, see Section 6).

D4. Single Linux host. Settings are read at start-up; the resolver choice and the demo fault injection can change while running.

2.2 Functional requirements

F1	Serve DoT (RFC 7858) on a TCP port to many concurrent clients, with several pipelined queries per connection.
F2	Validate every query and answer with the correct response code: FORMERR, NOTIMP, BADVERS, REFUSED, SERVFAIL.
F3	Resolve through a configurable chain: local records file, shared cache, forwarding (plain or **DoT**) or recursion from the root servers.
F4	Stay responsive under load: bounded queues, coalescing of duplicate lookups, load balancing over workers, and a per-request deadline.
F5	Refuse queries for blocked domains and their subdomains.
F6	Offer an operator view: live metrics, query log, upstream TLS evidence, and built-in tests.

2.3 Security requirements

S1 **Confidentiality, client hop.** TLS 1.3 only, with ephemeral ECDHE (forward secrecy) and AEAD ciphers. No plaintext DNS on the client link.

S2 **Confidentiality and authenticity, upstream hop.** A `tls://` upstream is used only after its certificate passes: chain to a trusted CA, issued to the configured name (or IP), within its validity dates. A failed check is a failure, never a downgrade to plain DNS (RFC 8310 strict profile).

S3 **Size hiding.** When the client asks for EDNS padding, replies are padded to 468-byte blocks; queries to DoT upstreams are always padded to 128-byte blocks, which also asks the upstream to pad its replies (RFC 7830/8467).

S4 **Input safety.** The parser never reads outside the message. Malformed input yields FORMERR or a closed connection, never a crash.

S5 **Response integrity.** Upstream replies must match the query ID and question; only records belonging to the question are cached; recursion enforces bailiwick.

S6 **Availability.** Per-client token-bucket rate limiting; strike-based blocking before the TLS handshake; connection and per-source limits; bounded queues; deadlines; circuit breakers.

S7 **Management plane.** The dashboard is reachable only on loopback, and the HTTP Host header is checked to stop DNS-rebinding attacks.

2.4 Assets, trust boundaries and adversaries

Assets: the names users look up (privacy); the integrity of answers; the availability of the resolver; the server's TLS private key; the cache contents.

Trust boundaries (marked TB1–TB3 in Figure 1):

- **TB1**, client network to resolver: everything arriving here is untrusted bytes.
- **TB2**, resolver to upstream network: replies are untrusted until the TLS peer is authenticated and the reply matches the question.
- **TB3**, operator to dashboard: loopback only.

Table 1: Adversaries and attack scenarios

Adversary	Attack scenario	Mitigation (requirement)
Passive on-path observer (Wi-Fi, ISP)	Reads names from plaintext DNS, or infers them from message sizes	DoT on both hops; padding (S1–S3)
Active on-path attacker / impostor upstream	Intercepts port 853 with its own certificate; or blocks 853 hoping for a fallback to port 53	Certificate validation; no plaintext fallback (S2)
Off-path spoofer	Races forged replies to poison the cache	Random ID and source port; reply must match ID and question; cache only the question's own records; bailiwick (S5)
Malicious or buggy client	Malformed or oversized messages, compression loops, zero-length frames, non-TLS bytes	Bounds-checked parser, framer limits, protocol strikes (S4, S6)
Flooding client	Thousands of queries per second; many connections; slowloris trickling	Token bucket; blocking before TLS; connection caps; idle timeout counts only whole messages (S6)
Failing or hostile upstream	Never answers, answers garbage, or closes connections	Per-upstream timeout within the client deadline; reply validation; failover; circuit breakers (S5, S6)
Web page in the operator's browser	DNS rebinding to drive the admin API	Loopback bind and Host check (S7)

2.5 Out of scope: what the prototype does not protect

- **Upstream honesty.** No DNSSEC validation: a trusted upstream that lies is believed. Certificate validation proves *who* answered, not that the answer is true.
- **Privacy from the resolvers themselves.** Our server and the upstream necessarily see the names.
- **Metadata.** IP addresses, timing, the number of queries, and the upstream's SNI and IP remain visible.
- **Volumetric DDoS** beyond one host's capacity, and attacks on the host OS.
- **Other transports.** DoH and DoQ are not implemented. Recursion uses plain DNS to authoritative servers, as every recursive resolver must today.
- A compromised resolver host, or a compromised CA in the trust store.

3 Architecture and design

3.1 Components and interfaces

The three layers meet at two narrow interfaces, which let each layer be built and tested alone. The middleware was developed against a stub resolver, and the front end against a fake middleware.

```
// Front end -> middleware. Never blocks, never throws; cb runs exactly once, on any thread.
void Middleware::submit(std::string_view name, RecordType type, ClientInfo client, ResultCallback cb) noexcept;
bool Middleware::is_blocked(const ClientInfo&) const;          // checked at accept() and before each query

// Middleware -> backend. Blocking; may throw (the worker turns it into SERVFAIL). One chain per worker.
class Resolver { virtual ResolveResult resolve(const DomainName&, RecordType, RecordClass, bool recurse = true) =
      0; };
```

Threads. One epoll I/O thread owns every client socket and TLS session. N worker threads (default 4) each run one blocking resolve at a time from their own bounded queue. One deadline-timer

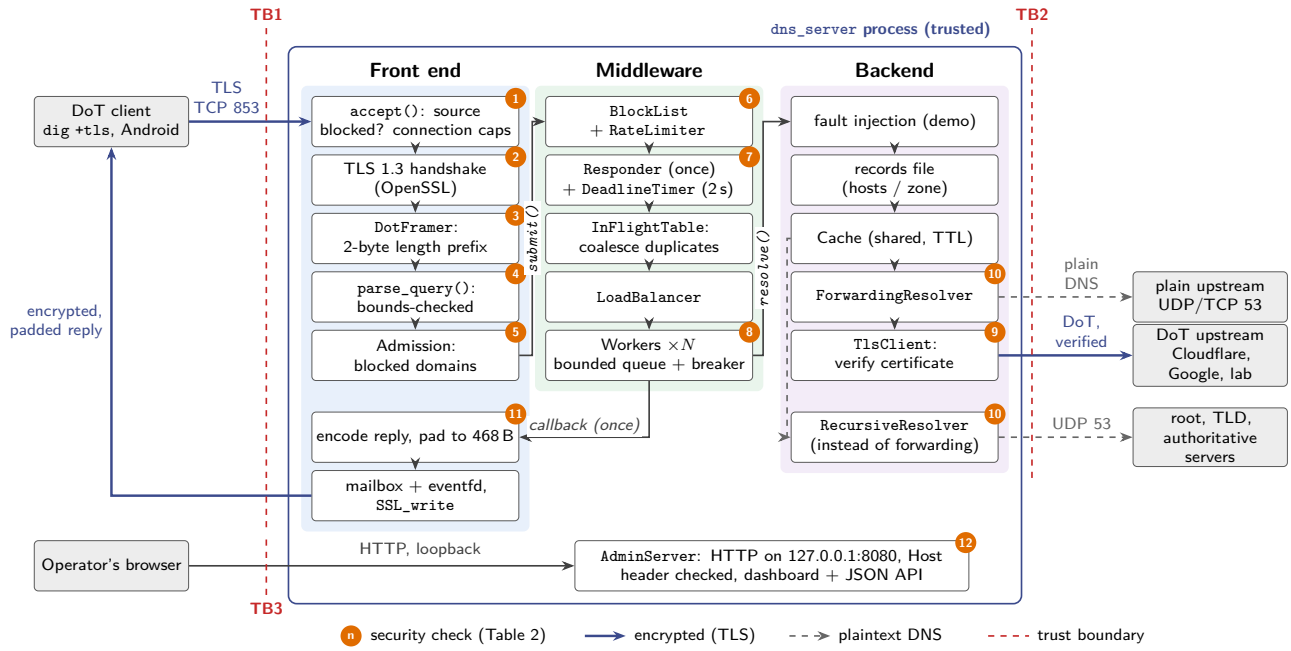


Figure 1: Architecture, end-to-end data flow and trust boundaries. Orange numbers mark the security checks listed in Table 2. Every worker owns its own resolver chain; only the cache is shared.

thread runs a min-heap of deadlines. The admin server uses one accept thread plus one thread per request or test. Client I/O is many idle connections, which suits one event loop. Resolving is few, latency-bound calls, which suits a thread pool. A queue separates the two, so a slow upstream can never stall the socket loop.

3.2 Life of one query (end-to-end data flow)

1. **Accept.** A blocked source gets a TCP reset before any TLS work. Total (1000) and per-source (32; IPv6 per /64) connection limits apply here.
2. **Handshake.** TLS 1.3 is negotiated without blocking. ALPN `dot` is confirmed if offered. Non-TLS bytes or an old TLS version earn a *protocol strike*.
3. **Frame.** The decrypted stream is cut into messages by the 2-byte length prefix. Pipelined messages are handled in order. A zero-length frame closes the connection with a strike.
4. **Parse.** The header, counts, name syntax, type, class and EDNS are checked. The result is FORMERR, NOTIMP or BADVERS on failure. A message too broken to carry an ID, or a response sent as a query, closes the connection.
5. **Policy.** A name on the blocked-domain list, or under it, is answered REFUSED. There is no strike, because asking is not abuse.
6. **Admit.** The middleware checks the block list and the client's token bucket. Over the rate means REFUSED plus a rate-limit strike. 50 strikes in 10 s block the source for 60 s.
7. **Deadline and coalesce.** A `Responder` takes ownership of the callback and the deadline is armed. If the same (name, type) is already being resolved, the query just waits for that lookup.
8. **Queue.** The load balancer (least-outstanding by default) picks a worker with room whose breaker is not open. If none can take the job, the client gets an immediate SERVFAIL "overloaded".
9. **Resolve.** The worker runs its chain: records file, then cache, then forwarding or recursion. For a `tls://` upstream, `TlsClient` reuses a verified connection to the same address *and* name, or opens a new one and verifies the certificate before writing the question.
10. **Reply.** Each waiter gets the result, or SERVFAIL if the deadline fired first; the atomic `done` flag lets only one win. The callback encodes and pads the reply and posts it to the I/O thread's mailbox, which encrypts and sends it.

3.3 Where security checks occur

Table 2: Security checkpoints (numbers as in Figure 1)

#	Where	Check	On failure
1	`DotServer` accept	Source blocked? Total and per-source connection caps	TCP reset, no TLS
2	`TlsContext`	TLS 1.3 only (ECDHE, AEAD); key file permissions warned at start	Handshake fails, strike
3	`DotFramer`	Length prefix; zero-length; idle timeout counts only whole messages; 256 KiB unsent cap	Close, strike
4	`parse_query`	Every read bounds-checked over `std::span`; compression loop and length limits; strict name syntax	FORMERR / NOTIMP / BADVERS
5	`Admission`	Blocked domain or parent of the name	REFUSED
6	`RateLimiter`, `BlockList`	Token bucket per source; strikes in a window lead to a timed block	REFUSED; later reset
7	`DeadlineTimer`	Every query answered within `request_timeout_ms`	SERVFAIL "timed out"
8	Workers, LB	Bounded queues; breaker opens after 3 failures in a row; expired jobs skipped	SERVFAIL "overloaded"
9	`TlsClient`	Chain to trusted CA; `SSL_set1_host` name or IP check; validity dates; TLS $\geq$ 1.2; question sent only after success	Next upstream, else SERVFAIL; never plain
10	`Upstream`, `Cache`, recursion	Reply ID and question match; only the question's own chain cached; bailiwick; per-question budget (64 queries, depth 8)	Reply ignored or SERVFAIL
11	Encoder	Replies padded to 468 B blocks; queries to DoT upstreams to 128 B; on encoding failure, fall back to a valid SERVFAIL	—
12	`AdminServer`	Loopback bind; Host header must be loopback	HTTP 403

3.4 Error handling

Expected failures are returned as `std::expected<T,E>`: a bad configuration, a malformed query or a TLS set-up error comes back with a precise message, for example `line 7: unknown key 'worker'`. Impossible states throw at construction. The whole query path sits behind `noexcept` boundaries that turn anything thrown into SERVFAIL, so one bad query can never take the server down or lose a reply. Configuration is fully validated at start-up (ranges, file existence, certificate/key pairing). A server that starts is correctly configured.

Table 3: Rejection behaviour seen by a client

Situation	Reply	Reason
Blocked client; message too broken to read; zero-length frame	close / reset	Nothing valid to answer; costs us nothing
Malformed query	FORMERR (1)	Tells a buggy client what is wrong
Unsupported opcode, class (CHAOS), type (AXFR)	NOTIMP (4)	Clear answer instead of silence
EDNS version > 0	BADVERS (16)	Client retries with version 0 and keeps padding
Blocked domain; over the rate limit	REFUSED (5)	Valid but not allowed; "don't retry"
Overload, timeout, upstream failure, certificate rejected	SERVFAIL (2)	Resolver-side problem; retry later

3.5 Key design choices and reasoning

Choice	Reasoning
DoT rather than DoH	DNS's own wire format inside TLS on a dedicated port. No HTTP/2 stack needed, and `dig`, `kdig`, systemd-resolved and Android support it natively. Same privacy for the payload.
TLS 1.3 only toward clients	Every 1.3 suite has forward secrecy and AEAD; no downgrade paths to configure.
Strict upstream validation, no fallback	Falling back to port 53 would let anyone who blocks port 853 downgrade every query to plaintext (RFC 8310).
Keep verified upstream connections (64, 10 s idle)	A cold DoT query costs TCP + TLS handshakes (three round trips). Reuse brings it to one, like UDP. Reuse is keyed on address *and* name, so a connection verified for one name is never used for another.
Raw epoll + OpenSSL, no Asio/libuv	OpenSSL is the only dependency, and every TLS state (`WANT_READ/WRITE`, pause, reset) is visible and testable.
Blocking resolvers on a worker pool	Forwarding and recursion stay plain synchronous code that is easy to test. The cost is throughput on cache misses (Section 6).
Exactly-once `Responder`	An answer and the deadline race on different threads. One `atomic<bool>::exchange` decides who replies, without locking the hot path.
Coalescing on normalised (name, type)	A popular name expiring under 500 concurrent clients becomes one upstream query, not 500.
REFUSED for rate limits	REFUSED tells a client not to retry; SERVFAIL would invite more traffic.
Block before TLS	The handshake is the most expensive thing a client can make us do. A blocked client should cost one `close()`.

4 Implementation

4.1 Language, libraries and environment

- **Language:** C++23 (GCC 15), built with CMake presets `local`, `release`, `asan` (AddressSanitizer + UBSan) and `tsan` (ThreadSanitizer), with `-Wall -Wextra -Wpedantic`.
- **C++23 features used:** `std::expected` for errors on expected paths; `std::format`; `std::jthread` / `stop_token` for threads that shut down cleanly; `std::move_only_function` for callbacks.
- **Run-time dependency:** OpenSSL 3 (libssl) only. GoogleTest is fetched by CMake for tests.
- **Environment:** Linux (epoll, eventfd, POSIX sockets), tested on Ubuntu 22.04, kernel 6.8, and Fedora 44, kernel 7.2.
- **Tools:** `dig` 9.18 (`+tls`), `openssl s_client`, `dumpcap`/`tshark`, Python for hand-crafted malformed messages, Chrome for the dashboard.

4.2 Modules and reuse

Table 4: Modules by layer (all written by the team; external code is listed separately)

Layer	Files	Responsibility	Lines
Front end	`dot_server`, `tls_context`, `dot_framer`, `dns_message`, `admission`, `main`	DoT server, TLS, framing, parser/encoder, padding, blocked domains	≈1,900
Middleware	`middleware`, `responder`, `deadline_timer`, `inflight_table`, `worker`, `load_balancer`, `rate_limiter`, `block_list`, `bounded_queue`, `metrics`, `query_log`, `config`, `resolver_factory`, `admin_server`, `gui/index.html`, `tools/load_gen`	Admission control, scheduling, reliability, configuration, observability, dashboard	≈5,300
Backend	`backend/`: `Cache`, `Upstream` (incl. `TlsClient`), `ForwardingResolver`, `RecursiveResolver`, `HostsFileResolver`, `ZoneFilesResolver`; `DomainName`, `IP`, `Records`	Resolution, caching, upstream transport and certificate checks, zone files	≈2,700
Tests	`tests/` (31 files + fakes)	534 unit, integration and end-to-end tests	≈8,200

Reused, not written by us: OpenSSL (TLS, X.509 verification, ECDHE); GoogleTest/GoogleMock; the C++ standard library; the Linux kernel's TCP stack. The DNS parser, encoder, cache, resolvers, middleware, DoT server, test fakes (fake DNS server, fake DoT upstream, a throw-away CA issuing certificates with arbitrary names and dates) and the dashboard are our own.

4.3 Important algorithms

- **Token bucket per client** (`RateLimiter`). Refill = $\min(burst, tokens + qps \cdot \Delta t)$, one token per query. Idle buckets are swept every 1024 calls, so an attacker cycling addresses cannot grow memory without bound.
- **Deadline min-heap** (`DeadlineTimer`). One thread sleeps until the earliest `(deadline, weak_ptr<Responder>)`. Answered queries are freed at once, and callbacks run outside the lock to avoid deadlock on re-entry.
- **Circuit breaker.** Closed → Open after `failure_threshold` consecutive SERVFAILs/exceptions; Open → Half-open after `cooldown_ms`, admitting one probe. Success closes it; failure reopens it. NXDOMAIN and "expired in queue" do not count.
- **Least-outstanding load balancing.** Fewest queued jobs, then lowest EWMA latency, then id. It always returns a fallback worker, so one full queue does not fail a query.
- **Upstream timeout budgeting.** Per-upstream timeout = request timeout ÷ number of upstreams, clamped to 100–1000 ms, so every upstream can be tried inside the client's deadline. The last upstream that answered is shared across workers and asked first, so a dead upstream costs one timeout, not one per worker.
- **Certificate verification** (`TlsClient`). `SSL_VERIFY_PEER` against the system store or `upstream_ca_file`; `SSL_set1_host` (no partial wildcards) with SNI, or `X509_VERIFY_PARAM_set1_ip_asc` when no name is configured; minimum TLS 1.2. The connect, handshake, write and read all share the per-upstream deadline.
- **Iterative recursion** from the 13 root hints. It follows referrals and CNAMEs, enforces bailiwick on records and glue, shuffles server lists and skips lame servers, within a per-question budget.

4.4 Configuration and integration

All settings live in one `key = value` file, documented key by key in `config/middleware.example.conf`. The settings that matter for this report:

```
mode = forwarding
upstreams = tls://1.1.1.1#cloudflare-dns.com, tls://8.8.8.8#dns.google   # DoT, name checked
# upstream_ca_file = /path/ca.pem          # default: the system's trusted CAs
records_file = config/demo.hosts           # local names, answered first, never leaked upstream
blocked_domains_file = config/blocked_domains.txt
workers = 4                queue_capacity = 1024      request_timeout_ms = 2000
failure_threshold = 3      cooldown_ms = 3000         lb_policy = least_outstanding
rate_limit_qps = 200       rate_limit_burst = 100     block_rate_limit_strikes = 50
listen = 127.0.0.1:8853    tls_cert = config/server.pem   tls_key = config/server.key
admin_listen = 127.0.0.1:8080
```

`main` builds the resolver factory (one chain per worker plus the shared cache and `TlsClient`), the middleware, the DoT server and the admin server. On SIGINT/SIGTERM it stops them in reverse order: admin page, then front end (2 s grace for answers to go out), then middleware (drain, then SERVFAIL anything left).

4.5 Implementation challenges

- **Replies from any thread.** Callbacks run on workers, the timer or inside `submit()`, but only the I/O thread may touch an `SSL*`. We solved it with a mailbox and an `eventfd` watched by epoll. The mailbox is held by `shared_ptr` so a late callback after shutdown is still safe.
- **Non-blocking OpenSSL.** A retried write may come from a buffer that has since moved, which needs `SSL_MODE_ACCEPT_MOVING_WRITE_BUFFER`. Level-triggered epoll was required because OpenSSL can hold decrypted bytes that edge triggering would never report again.
- **Exactly-once under races.** The deadline and the answer may complete simultaneously, and a half-open breaker's probe can expire in the queue. Each case got its own test, run under ThreadSanitizer.
- **Stale kept connections.** Servers close idle DoT connections. If a reused connection fails before any reply byte arrives, a fresh one is opened within the same deadline, and a connection that carried a bad reply is dropped, never pooled.
- **Connection exhaustion.** Found in review: a client with unsent answers was exempt from the idle timeout, and trickled bytes counted as activity. Both fixed (Section 5.9, D2).

5 Testing, results and security evaluation

5.1 Test environment

- Ubuntu 22.04, Linux 6.8, GCC 15, OpenSSL 3.0.2, release build, `workers = 4`, deadline 2 s.
- **Network constraint.** The evaluation network (campus Wi-Fi) drops outbound TCP 853 to every public DoT resolver we tried (Cloudflare, Google, Quad9, AdGuard, OpenDNS, Mullvad, . . .) and plain DNS to all but 8.8.8.8:53. Instead of skipping the encrypted upstream hop, we built the *lab topology* of Figure 2: a second `dns_server` instance acts as a DoT upstream with a certificate from a private CA (`CN=resolver.lab.test`), and the main server trusts that CA via `upstream_ca_file`. The code path is identical to Cloudflare's; only the trust anchor differs. A third instance presents an *expired* certificate from the same CA. The unreachable public resolvers became live timeout tests.
- **Capture.** `dumpcap` inside an unprivileged network namespace (`unshare -rn`) recorded both hops on an isolated loopback. In the namespace, the lab upstream answers from its own records file.

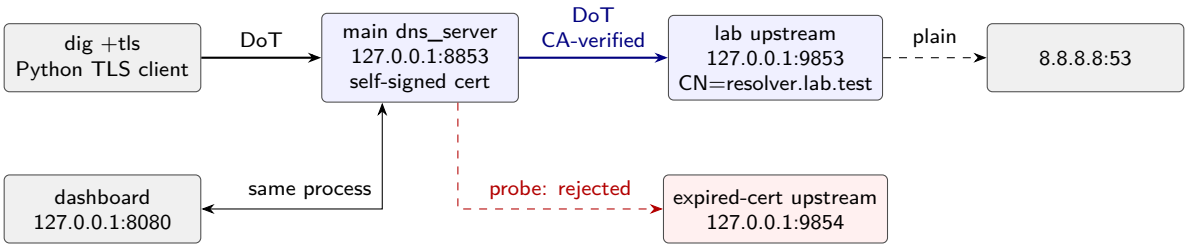


Figure 2: Lab topology used for the live evaluation. The configured upstream is `tls://127.0.0.1:9853#resolver.lab.test` with `upstream_ca_file` pointing at the lab CA.

5.2 Automated test suite

Table 5: Automated tests: 534 cases in 54 suites, 31 files. All pass in every build.

Build	Result	Time
Release build	**Pass**: 534/534	13.4 s
AddressSanitizer + UndefinedBehaviorSanitizer	**Pass**: 534/534, 0 reports	27.4 s
ThreadSanitizer	**Pass**: 534/534, 0 reports	15.4 s

Area	Representative suites	Tests
Wire format, input safety	DnsMessageParse/Encode/Upstream, DomainName, DotFramer, DotFrame, IP, Records	128
Front end over real TLS	DotServer, TlsContext, Admission	37
Middleware	Middleware, Responder, DeadlineTimer, InFlightTable, Worker, LoadBalancer, RateLimiter, BlockList, BoundedQueue, Metrics, QueryLog	97
Upstream DoT, certificates	DotUpstream (24), TlsClient (4), AdminTlsTest (4), MakeFactory (12), ParseUpstream (10)	54
Resolvers, cache, zones	ForwardingResolver (14), RecursiveResolver, Cache, ZoneFile, ZoneFilesResolver, HostsFile, FindRecords, StubResolver	161
Config, admin, misc.	ParseConfig, ParseListen, LoadConfig, AdminServerTest, AdminAbuseTest, small type tests	57

The upstream tests run against a **fake DoT server** whose certificate is issued on the fly by a throw-away test CA with any names, IPs and dates. Each rejection test also asserts that the fake server *received zero queries*: the question is never sent before the certificate passes.

5.3 Test cases: expected versus actual

Table 6: Key test cases (U = unit/integration test, L = live run on the lab topology). Evidence in the figures and listings below.

Type	Case	Expected	Actual	Result
Normal				
L	`dig +tls example.com A`	NOERROR over DoT, upstream over DoT	NOERROR, 2 A records, 11 ms; served by `upstream 127.0.0.1:9853 (resolver.lab.test) over TLS` (Lst. 1, Fig. 3)	**Pass**
L	Same name again	Answer from cache	0.09 ms, “cache: hit” (Fig. 3)	**Pass**
L	`nas.lan`, `homeassistant.lan AAAA`	Local records, never forwarded	192.168.1.20 / fd00::30 from “records file”	**Pass**

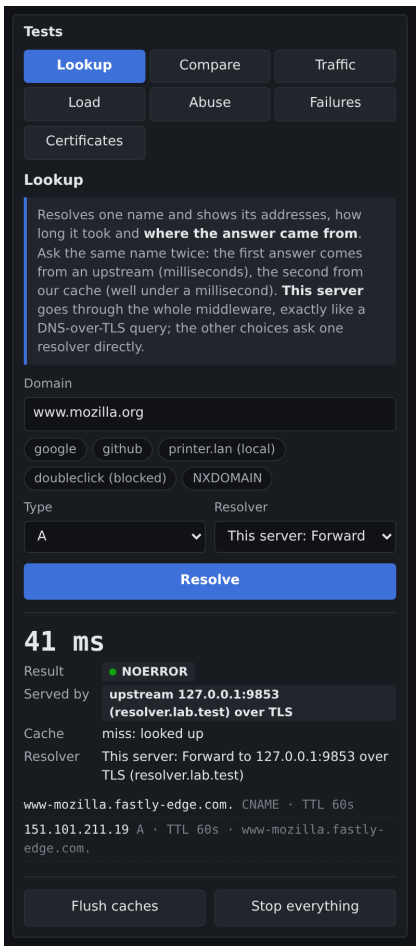
Type	Case	Expected	Actual	Result
L	`nonexistent-zzq.example.com`	NXDOMAIN passed on	NXDOMAIN	**Pass**
U	DotUpstream.AnswersOverTlsWithAVerifiedCertificate	Answer; SNI sent; subject/issuer recorded	As expected	**Pass**
U	DotUpstream.ConnectionIsKeptForTheNextQuery	Second query reuses the verified connection	1 handshake, 2 queries	**Pass**
U	DotUpstream.QueriesArePadded; PaddedQueriesAllHaveTheBlockSize	Every DoT upstream query carries the Padding option; 4- and 60-character names both encode to 128 B	As expected; plain UDP queries stay unpadded	**Pass**
L	Capture of the upstream hop	Upstream records the same size for every name	Queries 147 B, answers 487 B for all four names (Lst. 2)	**Pass**
Upstream certificate validation (negative)				
L+U	Wrong name: lab server checked as `dns.google`	Rejected, nothing sent	"hostname mismatch" (Fig. 4)	**Pass**
L+U	Expired certificate (9854)	Rejected	"certificate has expired"	**Pass**
L+U	Self-signed (our own 8853)	Rejected	"self-signed certificate"	**Pass**
L+U	No name, IP not in certificate	Rejected	"IP address mismatch"	**Pass**
U	Untrusted issuer; not yet valid; wildcard `*.x` for `x`; system store vs. test CA	Rejected, zero queries received	All rejected, 0 queries	**Pass**
U	MakeFactory.TlsUpstreamWithTheWrongNameFails	SERVFAIL; `rejected = 1`; `upstream.queries() = 0`	As expected	**Pass**
U	MovesOnToTheNextUpstreamWhenACertificateIsRejected	Second upstream answers	NOERROR from the second	**Pass**
U	AConnectionIsOnlyReusedForTheSameName	New handshake for another name	New handshake	**Pass**
Invalid upstream responses				
U	Garbage reply (TLS)	SERVFAIL, connection dropped	SERVFAIL	**Pass**
U	Reply to another question (wrong ID)	Ignored, SERVFAIL	SERVFAIL	**Pass**
U	Truncated reply; reply cut short mid-frame	SERVFAIL	SERVFAIL	**Pass**
U	AFailedQueryDoesNotPoisonTheNextOne	Next query succeeds on a fresh connection	NOERROR	**Pass**
U	Junk datagrams before the real answer (UDP)	Junk ignored, real answer used	NOERROR	**Pass**
U	CachesOnlyTheQuestionsOwnChain	Unrelated records not cached	Not cached	**Pass**
Upstream failure and timeouts				
L	Resolver switched to Cloudflare DoT (853 filtered)	SERVFAIL within deadline	SERVFAIL at 1000 ms (per-upstream budget) (Lst. 4)	**Pass**
L	Lab upstream process killed	Fast SERVFAIL	SERVFAIL in 0–1 ms (connection refused)	**Pass**
L	Lab upstream restarted (recovery)	Service resumes, no restart of main	NOERROR 13 ms, 74 ms	**Pass**
U	SilentServerTimesOut (DoT)	SERVFAIL within deadline	SERVFAIL	**Pass**

Type	Case	Expected	Actual	Result
U	PlainDnsServerIsAFailureNotAFallback	Fails; no plain query sent	0 plain queries	**Pass**
U	ServerClosingAKeptConnectionIsNotAnError	Transparent reconnect	NOERROR	**Pass**
L	Compare: 8 resolvers on the filtered network	Unreachable ones fail in bounded time	1.5 s / 3.0 s / 5.0 s SERVFAIL; none hangs (Fig. 7)	**Pass**
Adversarial client input (Lst. 5)				
L	QDCOUNT=2; label past end; compression loop; space in label	FORMERR	FORMERR	**Pass**
L	Header shorter than 12 bytes; QR=1; zero-length frame	Close, no reply	Closed	**Pass**
L	Opcode STATUS; class CHAOS; type AXFR	NOTIMP	NOTIMP	**Pass**
L	TLS 1.2 client	Handshake refused	Alert 70 “protocol version”	**Pass**
L	Abusive client 2,000 q/s	Rate limited, then blocked	Blocked after 0.09 s; 15,831 refused (Fig. 10)	**Pass**
L	Polite client 50 q/s	Never refused	400/400 answered	**Pass**
Load and failure recovery				
L	load_gen 100k flood; 20% failures; overload	Exactly one reply per query (exit 0)	exit 0 in all three runs	**Pass**
L	Failures test: 60% injected, then 0%	Breakers open, then close	All 4 open (7 trips each), all closed after (Fig. 11)	**Pass**

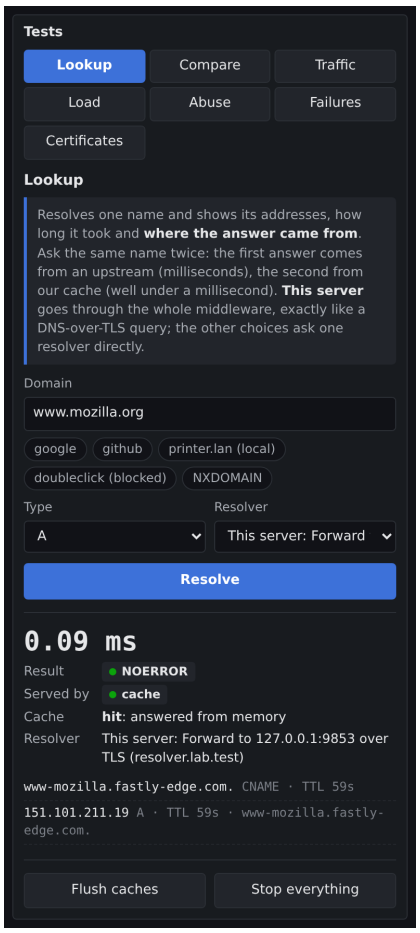
5.4 Evidence: a DNS query completing over DNS-over-TLS

Listing 1: `dig +tls` through the full chain (client DoT → server → DoT to the verified lab upstream)

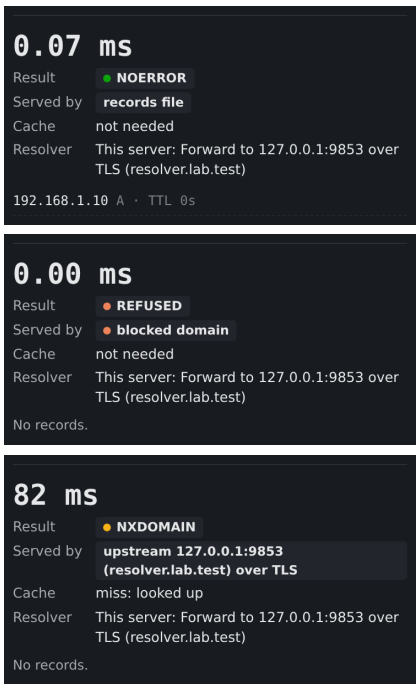
```
$ dig +tls @127.0.0.1 -p 8853 example.com A
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 5923
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1
; EDNS: version: 0, flags:; udp: 1232
;; ANSWER SECTION:
example.com.            66      IN      A       172.66.147.243
example.com.            66      IN      A       104.20.23.154
;; Query time: 11 msec
;; SERVER: 127.0.0.1#8853(127.0.0.1) (TLS)
```



(a) First lookup: miss, served by the DoT upstream.



(b) Second lookup: cache hit, 0.09 ms.



(c) Local record, blocked domain (REFUSED) and NXDOMAIN.

Figure 3: Dashboard Lookup test. "This server" sends the query through the whole middleware, like a DoT client. "Served by" names the hop that answered: `upstream 127.0.0.1:9853 (resolver.lab.test) over TLS`.

Packet capture of both hops. 83 packets were captured on the isolated loopback: four client connections to :8853 and one upstream connection to :9853, reused for all four names.

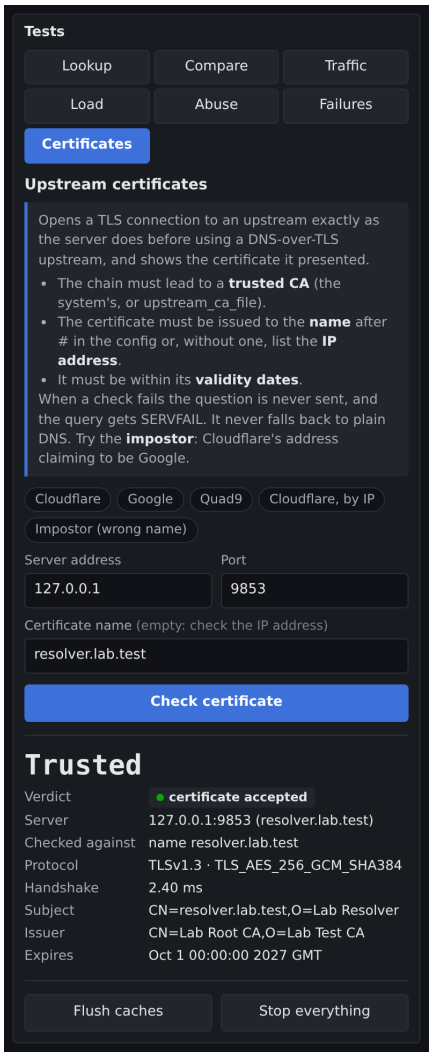
Listing 2: `tshark` on the capture: what an on-path observer sees

```
No.  src->dst      len  info
  4  40037->8853   369  Client Hello                      (ALPN "dot", TLS 1.3)
  6  8853->40037   879  Server Hello, Change Cipher Spec, Application Data x4
  9  8853->40037   321  Application Data
 10  40037->8853   558  Application Data                  query, padded: TLS record 487 B
 15  45352->9853   385  Client Hello                      SNI "resolver.lab.test"  (upstream hop)
 17  9853->45352   905  Server Hello, Change Cipher Spec, Application Data x4
 22  45352->9853   218  Application Data                  upstream query, padded:  TLS record 147 B
 23  9853->45352   558  Application Data                  upstream answer, padded: TLS record 487 B
 24  8853->40037   558  Application Data                  reply, padded: TLS record 487 B
 40  45352->9853   218  Application Data                  2nd name, same upstream connection (reused), 147 B
 41  9853->45352   558  Application Data                  its answer, 487 B
 ...
$ tshark -r dot.pcap -Y dns | wc -l                      ->  0   (no DNS message is readable)
$ strings dot.pcap | grep -ciE 'example|wikipedia|longer-name' ->  0   (no queried name appears)
TLS record sizes by name              client hop reply   upstream query   upstream answer
  example.com                                  487 B            147 B            487 B
  www.wikipedia.org                            487 B            147 B            487 B
  a-much-longer-name-...example.org            487 B            147 B            487 B
  example.com, client asked no padding          86 B            147 B            487 B
```

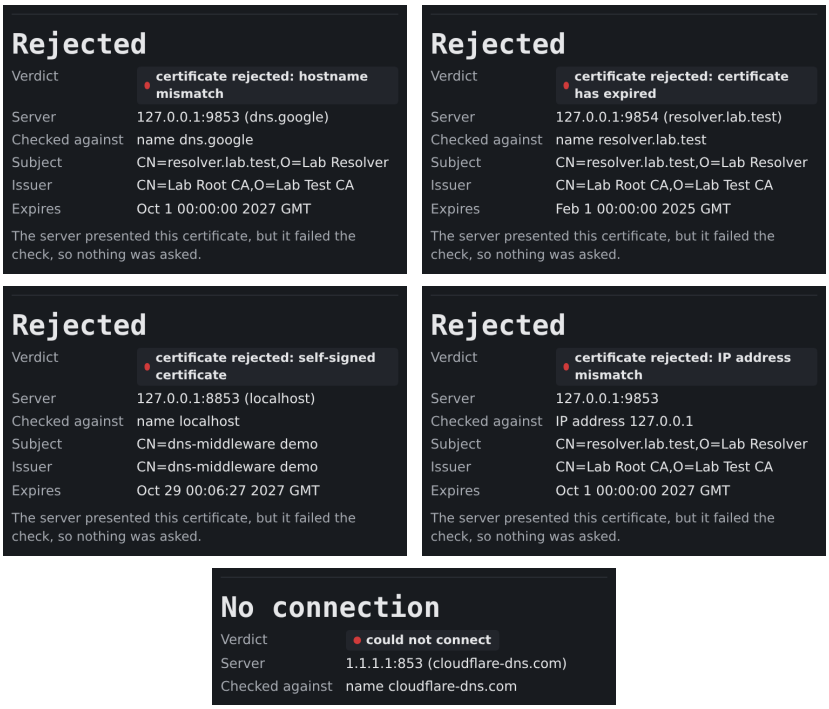
Listing 3: Client-facing TLS parameters and the server key's protection

```
$ openssl s_client -connect 127.0.0.1:8853 -alpn dot -brief
Protocol version: TLSv1.3        Ciphersuite: TLS_AES_256_GCM_SHA384
Server Temp Key: X25519, 253 bits          (ephemeral ECDHE -> forward secrecy)
$ openssl s_client -connect 127.0.0.1:8853 -tls1_2
... tlsv1 alert protocol version ... SSL alert number 70     (TLS 1.2 refused)
$ ls -l config/server.key ; git check-ignore -v config/server.key
-rw------- 1 xyz xyz 241 config/server.key
.gitignore:7:*.key   config/server.key
```

5.5 Evidence: upstream certificate validation



(a) Accepted: chain to the lab CA, name `resolver.lab.test`, TLS 1.3.



(b) Rejected: hostname mismatch (impostor), expired, self-signed, IP address mismatch; and "could not connect" for Cloudflare on the filtered network. Each says "nothing was asked".

Figure 4: Dashboard Certificates test: one real handshake per probe, made by the same `TlsClient` the resolver uses.

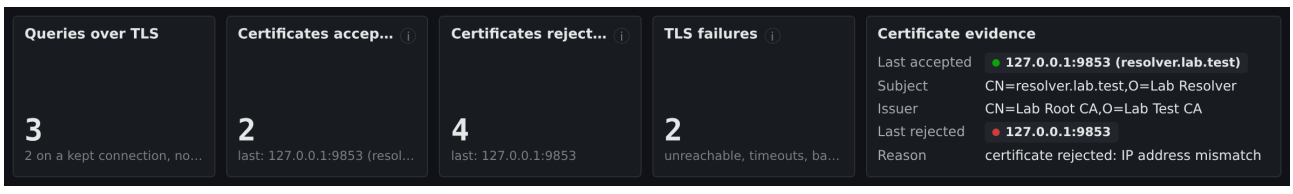


Figure 5: "Upstream encryption" row after the probes: queries over TLS (2 of 3 on a kept connection), certificates accepted and rejected, the accepted certificate's subject and issuer, and the last rejection's reason.

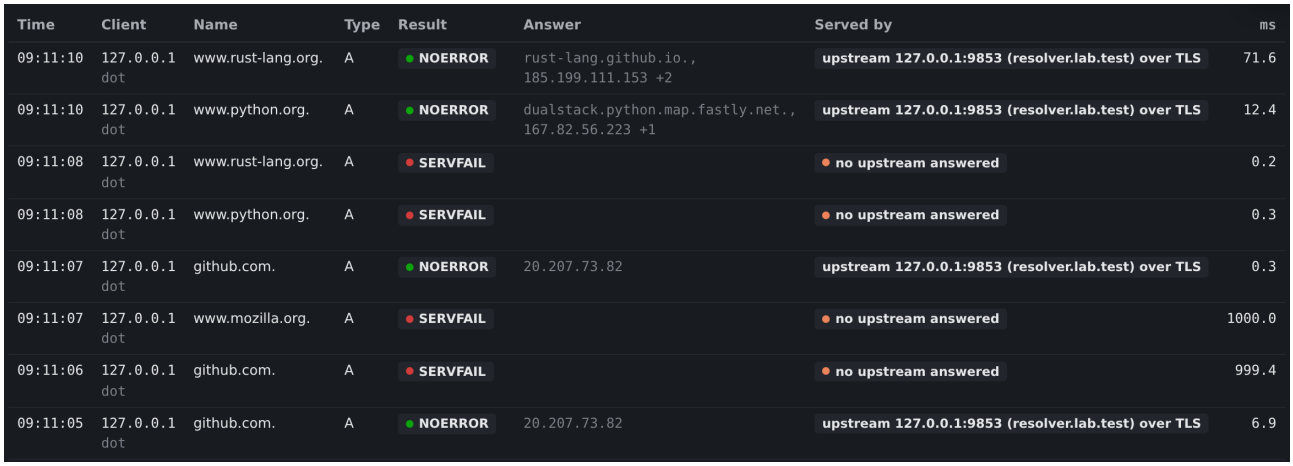
The same checks from the command line, independent of our code:

```
$ openssl s_client -connect 127.0.0.1:9853 -CAfile ca.pem -verify_hostname resolver.lab.test -brief
Protocol version: TLSv1.3   Peer certificate: O = Lab Resolver, CN = resolver.lab.test
Verification: OK            Verified peername: resolver.lab.test
$ openssl s_client -connect 127.0.0.1:9854 -CAfile ca.pem -verify_hostname resolver.lab.test -
    verify_return_error
verify error:num=10:certificate has expired   notAfter=Feb  1 00:00:00 2025 GMT
```

5.6 Evidence: invalid responses, upstream failure, timeouts and recovery

Listing 4: Failure and recovery seen by a real DoT client (`dig +tls`, one try each)

```
## 1. Normal: configured DoT upstream (resolver.lab.test)
github.com           status: NOERROR   Query time: 9 msec
## 2. Timeout: resolver switched to Cloudflare over TLS (port 853 filtered on this network)
github.com           status: SERVFAIL  Query time: 1001 msec
www.mozilla.org      status: SERVFAIL  Query time: 1000 msec
## 3. Back to the configured upstream
github.com           status: NOERROR   Query time: 1 msec
## 4. Upstream failure: the lab DoT upstream process is stopped
www.python.org       status: SERVFAIL  Query time: 0 msec
www.rust-lang.org    status: SERVFAIL  Query time: 1 msec
## 5. Recovery: the upstream is started again (main server untouched)
www.python.org       status: NOERROR   Query time: 13 msec
www.rust-lang.org    status: NOERROR   Query time: 74 msec
```



Time	Client	Name	Type	Result	Answer	Served by	ms
09:11:10	127.0.0.1 dot	www.rust-lang.org.	A	NOERROR	rust-lang.github.io., 185.199.111.153 +2	upstream 127.0.0.1:9853 (resolver.lab.test) over TLS	71.6
09:11:10	127.0.0.1 dot	www.python.org.	A	NOERROR	dualstack.python.map.fastly.net., 167.82.56.223 +1	upstream 127.0.0.1:9853 (resolver.lab.test) over TLS	12.4
09:11:08	127.0.0.1 dot	www.rust-lang.org.	A	SERVFAIL		no upstream answered	0.2
09:11:08	127.0.0.1 dot	www.python.org.	A	SERVFAIL		no upstream answered	0.3
09:11:07	127.0.0.1 dot	github.com.	A	NOERROR	20.207.73.82	upstream 127.0.0.1:9853 (resolver.lab.test) over TLS	0.3
09:11:07	127.0.0.1 dot	www.mozilla.org.	A	SERVFAIL		no upstream answered	1000.0
09:11:06	127.0.0.1 dot	github.com.	A	SERVFAIL		no upstream answered	999.4
09:11:05	127.0.0.1 dot	github.com.	A	NOERROR	20.207.73.82	upstream 127.0.0.1:9853 (resolver.lab.test) over TLS	6.9

Figure 6: Live query log for Listing 4, newest first: success over TLS, SERVFAIL “no upstream answered” at the 1000 ms upstream budget, fast SERVFAIL while the upstream is down, and recovery.

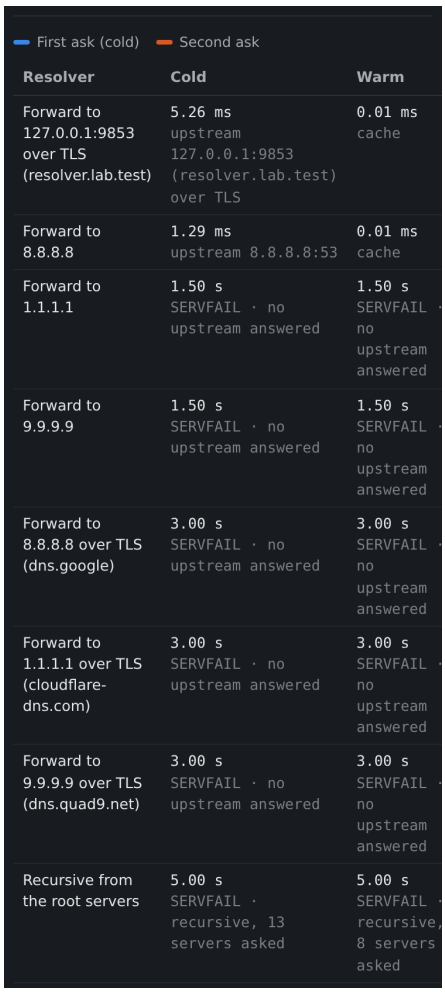


Figure 7: Compare test on the filtered network. Only the lab DoT upstream and 8.8.8.8 answer. Every unreachable resolver fails with SERVFAIL in bounded time: plain upstreams 1.5 s, DoT upstreams 3.0 s, recursion 5.0 s after asking 13 servers within its budget. These are direct calls that bypass the middleware. Through the middleware the 2 s deadline (or the 1 s per-upstream budget) applies, as Listing 4 shows.

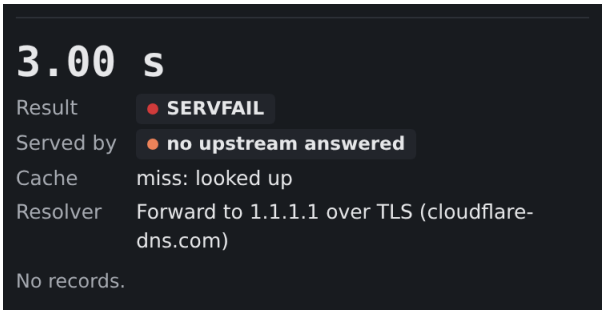


Figure 8: Lookup through Cloudflare over TLS: SERVFAIL "no upstream answered". It never falls back to plain DNS.

Listing 5: Hand-made malformed and unsupported DoT messages (Python `ssl` client)

```
valid query example.com A                    -> NOERROR (id 0x1234, 83 bytes)
truncated header (5 bytes)                   -> connection closed, no reply
QDCOUNT=2                                    -> FORMERR (id 0x1234, 12 bytes)
label length runs past the end               -> FORMERR (id 0x1234, 12 bytes)
compression pointer loop                     -> FORMERR (id 0x1234, 12 bytes)
bad name syntax (space in label)             -> FORMERR (id 0x1234, 12 bytes)
response bit set (QR=1)                      -> connection closed, no reply
opcode STATUS (2)                            -> NOTIMP (id 0x1234, 12 bytes)
class CHAOS                                  -> NOTIMP (id 0x1234, 12 bytes)
type AXFR (252)                              -> NOTIMP (id 0x1234, 12 bytes)
blocked domain ad.doubleclick.net            -> REFUSED (id 0x1234, 36 bytes)
zero-length frame                            -> connection closed, no reply
```

5.7 Evidence: availability under load and abuse

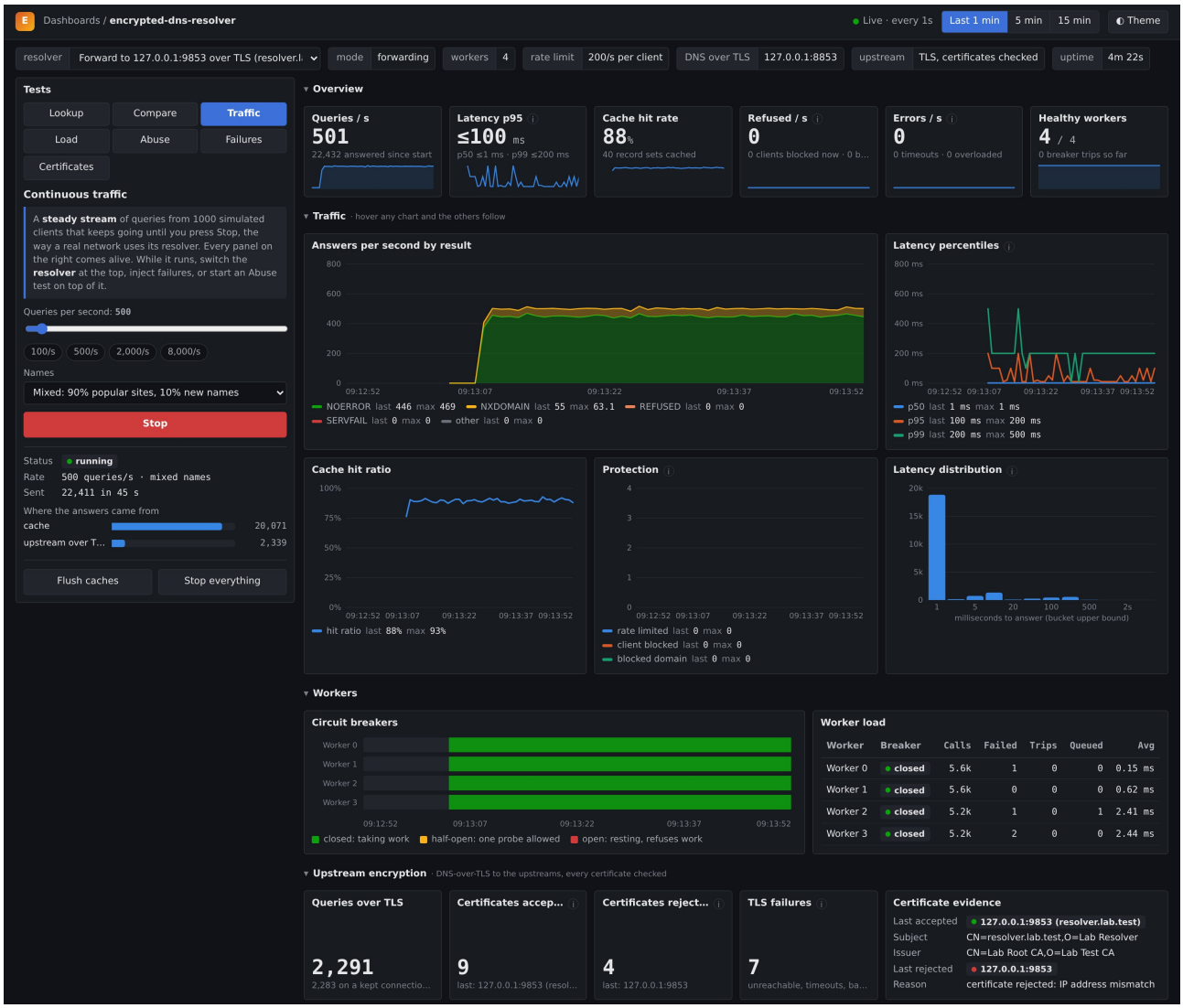
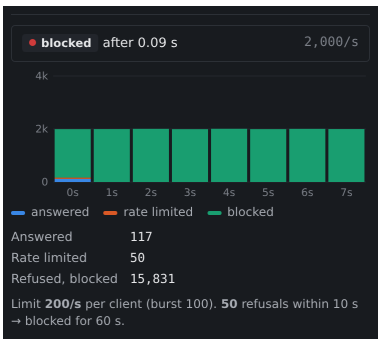
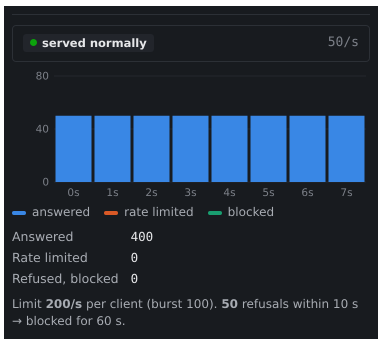


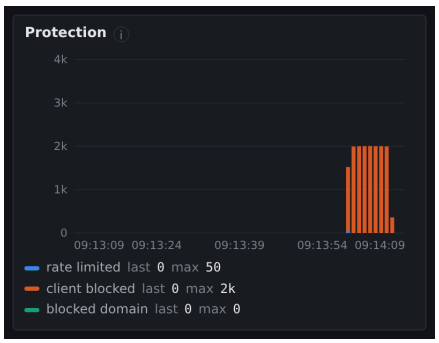
Figure 9: Dashboard (dark mode) during the Traffic test: 500 q/s from 1000 simulated clients, 90% popular and 10% new names, for 45 s. 22,432 answered, 0 errors, cache hit rate 88%, p50 ≤1 ms, p95 ≤100 ms; 2,339 answers from the DoT upstream, 2,283 of them on a kept connection; all four breakers closed.



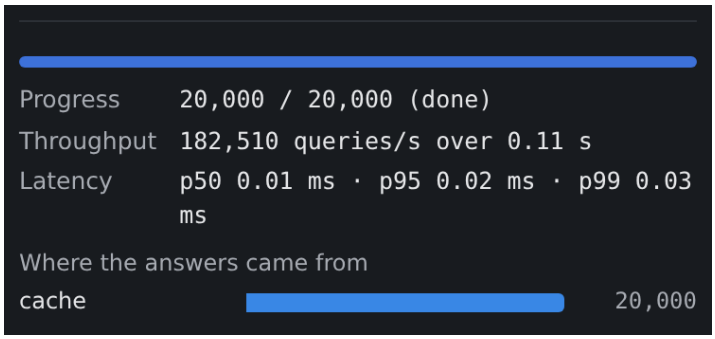
(a) 2,000 q/s: blocked after 0.09 s.



(b) 50 q/s: served normally.



(c) Protection panel: client blocked.



(d) Load test: 20k cached queries at 182k q/s.

Figure 10: Abuse and Load tests. The abusive client got 117 answers and 50 rate-limited replies, then 15,831 REFUSED while blocked. The polite client was never refused.

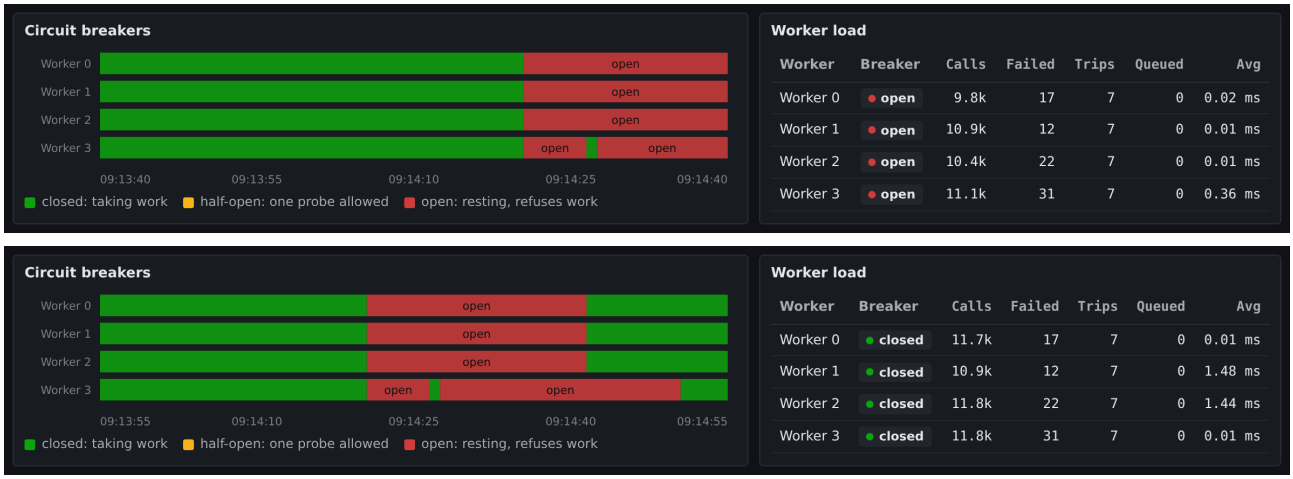


Figure 11: Failures test. Top: 60% of lookups fail inside the workers; after 3 failures in a row each breaker opens (red), so clients get a fast SERVFAIL. Bottom: with failures set back to 0%, half-open probes succeed and every breaker closes (green) without a restart.

Table 7: `load_gen`: middleware alone, stub resolver (≈5 ms), 8 client threads, 30% duplicate names

Run	Queries	Throughput	SERVFAIL	Coalesced	Outcome
Paced, 720 q/s (within capacity)	20,000	720 q/s	0	403	p50 5 ms, p99 10 ms; exit 0
Flood, unpaced	100,000	59,851 q/s offered	74,051	24,865	overload shed in ≤1 ms; every query 1 reply; exit 0
Flood + 20% resolver failures	100,000	57,934 q/s offered	75,913	24,727	breakers tripped; exit 0
1 worker, queue 1, 50 ms delay	2,000	—	mostly overload	—	exit 0

5.8 Security evaluation

Table 8: Security requirements against evidence

Req.	Evidence and assessment	Verdict
S1	TLS 1.3 only (TLS 1.2 refused), X25519 ECDHE, AES-256-GCM. The capture has 0 decodable DNS frames and 0 occurrences of queried names.	Met
S2	Lab: accepted only for the right name, CA and dates. Wrong name, expired, self-signed and wrong IP rejected live; untrusted, not-yet-valid and wildcard rejected in tests. Tests assert zero queries reach a rejected server, and a plain server is a failure, not a fallback.	Met
S3	Client hop: three names of very different lengths all give 487 B replies. Upstream hop: every query 147 B and every answer 487 B, whatever the name, and even when the client asked for no padding. Before the fix for D4 the upstream records were 59–108 B and varied with the name.	Met
S4	123 wire-format tests plus live malformed messages: FORMERR or close, never a crash. Zero ASan/UBSan reports over the whole suite.	Met
S5	Wrong-ID, wrong-question, garbage and junk-datagram tests; cache-only-own-chain test; bailiwick tests in recursion.	Met
S6	Abusive client blocked in 0.09 s; polite client unaffected; flood answered exactly once per query; breakers open and close; every failure bounded by the deadline.	Met
S7	Admin bound to 127.0.0.1; Host header checked (AdminServerTest).	Met

What the evaluation does not show.

- The public-resolver path (Cloudflare, Google) could not be exercised live on this network. It uses the same `TlsClient` with the system CA store; the only difference from the lab path is the trust anchor.
- Padding hides the reply's size, not its timing, nor the number of queries per page load.
- SNI on the upstream hop names the resolver being used, not the query.

5.9 Defects discovered

ID	Defect	Status
D1	`DomainName` from wire: wrong size bound, two off-by-ones (next offset, compression pointer), missing overflow check before copying a label. Found by parser tests.	Fixed (6f7c47d)
D2	Connection exhaustion: unsent answers exempted a connection from the idle timeout; trickled bytes counted as activity; no per-source connection cap.	Fixed (9825c9a), one test per fix
D3	Flaky tests: per-test temp directories collided between processes; a fake-clock jump timed out a running lookup; a connect/reset race.	Fixed (9825c9a)
D4	Queries to DoT upstreams carried no EDNS padding, so their size (59–108 B) leaked the name's length to an observer of the upstream hop. Found in this evaluation's capture. Fixed by padding every `tls://` query to 128 B, which also makes the upstream pad its answers.	Fixed (`Upstream`, `encode_query`), 3 new tests
D5	Dashboard sidebar clipped long "Served by" values: the grid cell lacked `min-width:0` and badges never wrapped. Found while taking this report's screenshots.	Fixed (`gui/index.html`)
D6	Old sanitizer build directories configured with g++-14 fail to compile (`append_range`). Build configuration only; fresh g++-15 configure passes.	Workaround: reconfigure
D7	The final merge (f38d405) brought back `std::ranges::ends_with` in `DomainName`, which GCC 15's standard library lacks, so `main` no longer compiled. Found when re-running the suite for this report.	Fixed (helper restored)

6 Limitations and lessons learned

6.1 Achieved requirements

F1–F6 and S1–S7 are met; the one gap found during evaluation (upstream padding, D4) was fixed. Compared with the Stage I plan we added three things: an *encrypted, certificate-checked upstream hop* (Stage I listed it as a non-goal), a full recursive resolver with zone files, and a live dashboard with built-in tests.

6.2 Known weaknesses

Limitation	Effect
Blocking resolvers	Cache-miss throughput ≈ workers × (1000 ms ÷ upstream RTT): 100–150/s with 4 workers. Above that, queries get SERVFAIL "expired in queue". This is deliberate load shedding, but it limits capacity.
Cache checked inside the worker	Under overload, a cacheable query still waits behind slow lookups.
No negative caching	NXDOMAIN is asked upstream every time.
Query IDs from `mt19937`	Not cryptographically random; the random source port and TLS mitigate this.
No DNSSEC; recursion over IPv4 only	Answers are trusted as received; IPv6-only zones unreachable.
No certificate pinning	Any trusted CA's certificate for the name passes.
Rate limit per IP (IPv6 /64)	Clients behind one NAT share a bucket and a block (false positives).

False positives and negatives. The blocked-domain list matches exact names and their subdomains only. A tracker on an unlisted domain passes (false negative), and blocking a parent blocks every child (possible false positive). Abuse blocking can hit honest users behind a shared NAT. The thresholds (200 q/s, 50 strikes in 10 s) are generous so that heavy browsing is never blocked: the "Heavy, 250/s" profile is rate-limited but a polite 50/s client is never refused.

Performance and usability. Cached answers take 0.01–0.1 ms. Upstream DoT costs one extra round trip only on a cold connection; afterwards 98% of DoT queries rode a kept connection (2,283 of 2,339). The dashboard explains every test in plain words and runs real traffic through the real middleware, which made the evaluation reproducible. Its live log was the fastest way to see why a query failed.

What the prototype does not guarantee. It does not guarantee that answers are true (no DNSSEC), privacy from the resolver operators, hiding of timing or traffic volume, or service beyond the configured capacity: excess load is refused, not queued.

6.3 Lessons learned

- **Narrow interfaces paid off.** Agreeing on `submit()` and `resolve()` in Stage I let three people work in parallel against stubs and fakes. Integration took one day.
- **"Exactly once" must be designed, not tested in.** Every way a query can end (answer, deadline, overload, exception, shutdown) goes through one atomic gate, and `load_gen` exits non-zero if any query gets other than one reply.
- **Fail closed, fail fast.** A SERVFAIL in 1 ms beats a hung client. Bounded queues, deadlines and breakers turned every failure we injected into a quick, honest answer.
- **Test the network you will demo on.** The campus network blocks port 853 entirely. A local CA and a second instance let us test the real code path, and the unreachable resolvers turned into timeout tests.
- **Capture the wire.** The packet capture confirmed confidentiality and also found D4, which no unit test was looking for. After the fix, the same capture proved it closed.

6.4 Five things to remember about this project

1. **Both hops are encrypted, and the transport is TLS over TCP.** Plain DNS on UDP port 53 lets anyone on the path read and forge every lookup. Clients reach the resolver with DNS-over-TLS (TCP 853), and the resolver uses DoT again to reach its upstream, so a query is never in clear text between the client and the server that answers it. TLS needs a reliable, ordered byte stream, which is why it runs over TCP rather than UDP. TLS 1.3 agrees a fresh key per connection with X25519 (forward secrecy) and encrypts with AES-256-GCM. The capture shows no readable DNS message and no queried name on either hop (Listing 2). The one exception is recursion, which must speak plain DNS to authoritative servers.
2. **Encryption is only useful if you check who is on the other end, and you never fall back.** Before sending a question upstream, `TlsClient` requires the certificate to chain to a trusted CA, to match the configured name (or IP) and to be inside its validity dates. If a check fails, the next upstream is tried and then the client gets SERVFAIL. There is never a retry on plain port 53, so an attacker who blocks or intercepts port 853 gains nothing. The impostor, expired, self-signed and wrong-IP probes were all rejected with zero queries sent (Figure 4).
3. **Encryption hides the content but not the size, so messages are padded.** AES-GCM output is exactly as long as its input plus a 17-byte overhead, and a reply's length says a lot about the name. EDNS padding (RFC 7830/8467) rounds every message up to a fixed block: 468 B for replies when the client asks for padding, and 128 B for every query we send upstream. Every name then gives a 487 B reply record and a 147 B upstream query record. Padding is an EDNS option, so on the client hop the client decides; on the upstream hop we are the client, so

we always pad. The capture is how we found that this was missing at first (D4).

4. **Every query gets exactly one reply, and failure is fast.** An answer, the 2 s deadline, an overloaded queue, an exception and shutdown can all race to finish the same query. One atomic flag in the `Responder` lets exactly one of them reply. Bounded queues, a per-upstream time budget and per-worker circuit breakers turn every failure into a quick SERVFAIL instead of a client left waiting, and the service recovers on its own when the upstream returns. `load_gen` checks the rule under a 100,000-query flood with injected failures (Table 7).
5. **Untrusted input is rejected as early and as cheaply as possible.** A blocked source is reset before the TLS handshake, the most expensive work a client can cause. Connection caps (1000 in total, 32 per source), a token bucket per client and a strike count turn floods into REFUSED and then a 60 s block. Every read of client bytes is bounds-checked, so a lying length field, a compression loop or a 5-byte header ends in FORMERR or a closed connection, never a crash. The 128 wire-format tests ran clean under AddressSanitizer.

7 Deployment and future work

7.1 Dependencies and build

On Ubuntu:

```
sudo add-apt-repository ppa:ubuntu-toolchain-r/test && sudo apt update
sudo apt install g++-15 cmake libssl-dev dnsutils         # CMake >= 3.21, OpenSSL 3
cmake --preset release && cmake --build --preset release  # dns_server, load_gen, dns_tests
```

On Fedora:

```
sudo dnf install g++ cmake openssl-devel bind-utils       # CMake >= 3.21, OpenSSL 3
cmake --preset release && cmake --build --preset release  # dns_server, load_gen, dns_tests
```

7.2 Run

```
./tools/make_demo_cert.sh                      # self-signed config/server.pem + .key (mode 600, git-ignored)
./build/release/dns_server config/demo.conf    # DoT on 127.0.0.1:8853, dashboard on http://127.0.0.1:8080/
dig +tls @127.0.0.1 -p 8853 example.com        # encrypted query
dig +tls @127.0.0.1 -p 8853 nas.lan            # local record
dig +tls @127.0.0.1 -p 8853 ad.doubleclick.net # REFUSED (blocked domain)
```

`config/demo.conf` forwards over DoT to `tls://1.1.1.1#cloudflare-dns.com` and `tls://8.8.8.8#dns.google` using the system CA store. On a network that blocks port 853, pick a plain resolver in the dashboard's resolver switch, or use a lab upstream as in Figure 2:

```
upstreams = tls://127.0.0.1:9853#resolver.lab.test
upstream_ca_file = /path/to/lab-ca.pem
```

For a real deployment: listen on `0.0.0.0:853` (needs `CAP_NET_BIND_SERVICE`), use a CA-issued certificate, and keep `admin_listen` on loopback.

7.3 Tests

```
ctest --test-dir build/release             # or ./build/release/dns_tests   (534 tests)
cmake --preset asan && cmake --build --preset asan && ctest --preset asan
cmake --preset tsan && cmake --build --preset tsan && ctest --preset tsan
./build/release/dns_tests --gtest_filter='DotUpstream*:TlsClient*:AdminTls*'   # upstream DoT evidence
./build/release/load_gen                   # exit 0 = every query got exactly one reply
./build/release/load_gen --stub-fail-rate 0.2
```

7.4 Repository structure

```
include/            DomainName, IP, Records, dns_message (wire format), utils
include/mw/         middleware + front-end headers (namespace mw)
include/backend/    resolvers, Cache, Upstream / TlsClient
src/                library sources, main.cpp;  src/backend/  resolver sources
tests/              GoogleTest suite (31 files);  tests/support/  fake DNS / DoT servers, test CA, printers
tools/              load_gen.cpp, make_demo_cert.sh
gui/                index.html (dashboard, served by dns_server)
config/             demo.conf, demo.hosts, blocked_domains.txt, middleware.example.conf
docs/               frontend.md/.tex, this report
```

7.5 Most important next improvements

1. **Asynchronous resolution**: one thread keeping many upstream queries open (epoll or io_uring) and pipelining queries on a DoT connection (RFC 7766), so cache-miss throughput no longer scales with threads.
2. **Check the cache in `submit()`** before queueing, so cached names are served even under overload; add negative caching (RFC 2308).
3. **DNSSEC validation** in the recursive resolver, and optional SPKI pinning per upstream (RFC 7858 §4.2).
4. **DoH front end** calling the same `submit()`, for clients and networks that allow only port 443; cryptographic query IDs (`RAND_bytes`); configuration reload on SIGHUP.

8 Appendix: minimum technical evidence index

Project: **Encrypted DNS Resolver**. The chosen encrypted protocol is **DNS-over-TLS** (RFC 7858), used on both the client hop and the upstream hop.

Required evidence	What is shown	Where
A DNS query completing through the encrypted protocol	`dig +tls` gets NOERROR; the dashboard shows it served by the upstream "over TLS"; the capture shows only TLS 1.3 records on both hops, 0 DNS frames, no names, padded 487 B replies	Listing 1, Figure 3, Listing 2, Figure 9
Client-side TLS parameters	TLS 1.3, AES-256-GCM, X25519; TLS 1.2 refused; key file mode 600, git-ignored	Section 5.4 (`openssl s_client`)
Upstream certificate validation	Accepted certificate with subject, issuer, expiry, protocol; rejections for hostname mismatch, expired, self-signed, IP mismatch; "nothing was asked"; counters and last-rejection reason; `openssl` cross-check	Figures 4, 5; Section 5.5; Table 6
Tests for invalid responses	Garbage, wrong ID or question, truncated, cut mid-frame, junk datagrams, malformed TCP; no poisoning of the next query; only own-chain records cached	Table 6; `test_dot_upstream.cpp`, `test_forwarding_resolver.cpp`
Tests for upstream failure	Closed port; upstream killed → fast SERVFAIL → recovery on restart; plain server where TLS expected (no fallback); next upstream tried after rejection; breakers open and close	Listing 4, Figures 6, 11

Required evidence	What is shown	Where
Tests for timeouts	DoT upstream unreachable → SERVFAIL at 1000 ms; silent DoT server test; Compare test with all unreachable resolvers bounded	Listing 4, Figures 7, 6; `SilentServerTimesOut`
Malformed client input	FORMERR / NOTIMP / close for 12 hand-made messages	Listing 5
Availability	Abuse blocking, load results, exactly-once delivery	Figures 10, 9; Table 7
Whole suite, sanitizers	534/534 in release, ASan+UBSan, TSan	Table 5